

Group Activity 02

(3인 혹은 4인으로 팀을 구성하여 아래의 문제를 푼다. 팀 구성은 매 시간마다 달라져도 된다.)

팀원1: _____

팀원2: _____

팀원3: _____

팀원4: _____

1. 다음의 순환 함수의 반환값은? 시간복잡도는?

```
/* x는 양의 정수이다. */
int fun1(int x) {
    if (x < 2)    return 0;
    else if (x % 2 == 0)
        return 1 + fun1(x/2);
    else
        return 1 + fun1(x/2+1);
}
```

2. 다음의 순환함수의 반환값은? 이 함수를 $s=0$, $t=n-1$ 으로 호출할 경우에 시간복잡도는?

```
/* 배열 A에 A[0]에서 A[n-1]까지 n개의 정수가 저장되어 있다. s와 t는 임의의 정수이다. */
int fun2(int n, int A[], int s, int t) {
    if (n == 0)
        return 0;
    if (A[n-1] >= s && A[n-1] <= t)
        return 1 + fun2(n-1, A, s, t);
    else
        return fun2(n-1, A, s, t);
}
```

3. 다음의 순환함수가 결과적으로 하는 일은?

```
void fun3(int n)
{
    if (n == 0)
        return;
    fun3(n/2);
    printf("%d", n%2);
}
```

4. 다음의 순환함수가 결과적으로 하는 일은?

```
void fun4(char str[], int start, int end)
{
    if (start >= end)
        return;
    else {
        char temp = str[start];
        str[start] = str[end];
        str[end] = temp;
        fun4(str, start + 1, end - 1);
    }
}
```

5. 다음의 순환 함수의 반환값은? 시간복잡도는?

```
/* 배열 A에 A[0]에서 A[n-1]까지 n개의 정수가 오름차순으로 정렬되어 저장되어 있다. */
int fun5(int n, int A[]) {
    if (n == 1) return 1;
    int tmp = fun5(n-1, A);
    if (A[n-2] < A[n-1])
        tmp++;
    return tmp;
}
```

6. 다음 함수가 하는 일은? 시간복잡도는?

```
int fun6(int a[], int n)
{
    if(n == 1)
        return a[0];

    int x = fun6(a, n-1);
    return (x > a[n-1] ? x : a[n-1]);
}
```

7. 다음의 순환 함수의 반환값은? 시간복잡도는?

```
double fun7(double a[], int n)
{
    if (n==1) return a[0];
    else
        return (a[n-1] + (n-1)*fun7(a, n-1))/n;
}
```

8. 다음의 순환 함수의 반환값은? 시간복잡도는?

```
int fun8(int a, int b)
{
    if (b == 0)
        return 1;
    if (b % 2 == 0)
        return fun8(a*a, b/2);
    return fun8(a*a, b/2)*a;
}
```

9. 다음은 N-queen 문제를 푸는 순환함수이다. 이 함수를 변형하여 N-queen 문제의 서로 다른 해의 개수를 구하는 함수를 작성하라. **promising** 함수는 작성할 필요가 없다.

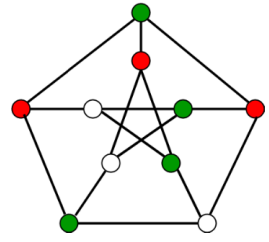
```
int cols[N+1];
bool queens( int level )    {
    if (!promising(level))
        return false;
    else if (level==N)
        return true;
    for (int i=1; i<=N; i++) {
        cols[level+1] = i;
        if (queens(level+1))
            return true;
    }
    return false;
}
```

10. 다음은 강의 슬라이드에 있는 미로에서 출구에 도달하는 경로가 존재하는지 검사하는 함수이다. 이 함수를 부분적으로 수정하여 출구까지 가는 서로 다른 경로의 개수를 카운트하여 반환하는 함수를 작성하라.

```
bool findMazePath(int x, int y) {
    if (x<0 || y<0 || x>=N || y>=N || maze[x][y] != PATHWAY_COLOUR)
        return false;
    else if (x==N-1 && y==N-1) {
        maze[x][y] = PATH_COLOUR;
        return true;
    }
    maze[x][y] = PATH_COLOUR;
    if (findMazePath(x-1,y) || findMazePath(x,y+1)
        || findMazePath(x+1,y) || findMazePath(x,y-1)) {
        return true;
    }
    maze[x][y] = BLOCKED_COLOUR;
    return false;
}
```

11. 입력으로 주어진 N 개의 정수들을 합이 동일한 두 집합으로 분할할 수 있는지 물어보는 문제를 PARTITION 문제라고 부른다. 이 문제를 해결하는 되추적기법 알고리즘을 고안하라. 상태공간트리를 기술하고 알고리즘을 말이나 pseudo 코드의 형태로 제시하라.

12. 하나의 그래프와 사용할 색의 개수 m 이 입력으로 주어진다. 그래프의 각각의 노드를 m 개의 색 중 하나로 칠하면서 어떤 인접한 두 노드도 동일한 색으로 칠해지지 않도록 할 수 있는지 검사하는 문제이다. 예를 들어 옆의 그림은 $m = 3$ 개의 색으로(red, green, white) 칠한 결과이다. 되추적(backtracking) 기법으로 이 문제를 해결하려고 한다. 적절한 상태공간트리를 구상하고 알고리즘을 pseudocode의 형태로 기술하라.



13. 평면상의 N 개의 점의 좌표가 입력으로 주어진다. 임의의 한 점에서 출발하여 모든 점들을 정확히 한 번씩 거쳐서 원래 출발점으로 돌아오는 가장 이동 거리가 짧은 경로를 찾는 문제를 TSP(Traveling Salesperson's Problem)이라고 부른다. 이 문제를 해결하는 되추적 알고리즘을 구상하여 pseudocode의 형태로 기술하라.